

天津大学

《智能搜索与问答》实验报告



Lab1 基于神经网络的命名实体识别

学 号 3021244141
姓 名 于梦洁
学 院 智能与计算学部
专 业 计算机科学与技术
年 级 2021 级
班 级 2 班

2024 年 3 月 16 日

目录

一、 数据准备与处理	3
二、 模型与算法介绍	3
1. 任务定义	3
2. BiLSTM-CRF 模型介绍	3
2.1 模型输入	4
2.2 LSTM 层	4
2.3 全连接层	5
2.4 CRF	5
2.5 模型整体结构	6
3. BiLSTM-CRF 模型优势	7
三、 实验流程	7
四、 实验结果	9
五、 结果分析	9
六、 心得体会	10

一、 数据准备与处理

在本实验中，为了便于理解，只使用了两条数据及其标签作为训练数据。

```
# Make up some training data
training_data = [(
    "the wall street journal reported today that apple corporation made money".split(),
    "B I I O O O B I O O".split()
), (
    "georgia tech is a university in georgia".split(),
    "B I O O O O B".split()
)]

word_to_ix = {}
for sentence, tags in training_data:
    for word in sentence:
        if word not in word_to_ix:
            word_to_ix[word] = len(word_to_ix)

tag_to_ix = {"B": 0, "I": 1, "O": 2, "START_TAG": 3, "STOP_TAG": 4}
```

图 1. 数据预处理相关代码

将训练数据中出现的所有词语放入字典中，字典的 key 为该词语，value 为编号，同时将 tag 也放入一个词典。

二、 模型与算法介绍

1. 任务定义

命名实体识别属于自然语言处理中的序列标注任务，是指从文本中识别出特定命名指向的词，比如人名、地名和组织结构名等，常见的两种标注方式为 BIO 与 BIOES。

	<START>	让	子	弹	飞	的	导	演	是	姜	文	;	刁	民	敢	杀	我	的	马	<END>
BIO	O	B	I	I	I	O	O	O	O	B	I	O	O	O	O	O	O	O	B	O
BIOES	O	B	I	I	E	O	O	O	O	B	E	O	O	O	O	O	O	O	S	O

图 2. 两种标注方法的示例

2. BiLSTM-CRF 模型介绍

对于命名实体识别，常用的模型为 BiLSTM-CRF。该模型使用 BIO 标注，加入 START 和 END 来使转移矩阵更加健壮，其中，START 表示句子的开始，END 表示句子的结束。这样，标注标签共有 5 个：[B, I, O, START, END]。

其中，模型的输入为字符特征，使用 LSTM 构建一个编码器，将输入序列映射到一个固定维度的特征空间。LSTM 可以捕捉到句子中的长距离依赖信息，帮助模型理解句子的语义。然后将 LSTM 的输出作为 CRF 的输入，利用 CRF 进行解码，计算出一个得分最高的标注序列，最后将其作为每个字符对应的预测标签

输出。

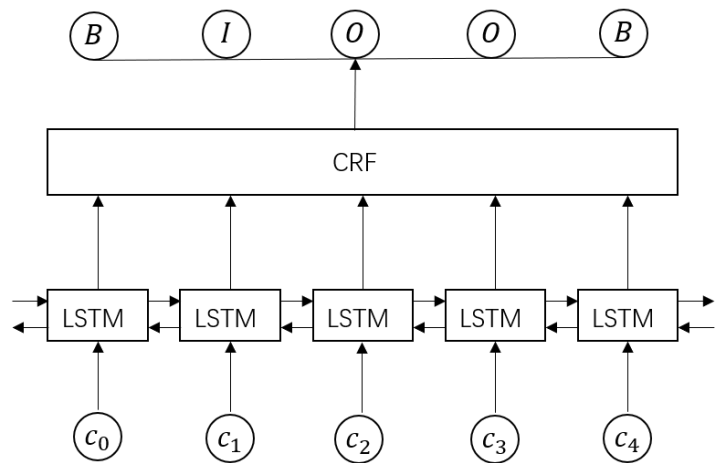


图 3. 模型整体框架

2.1 模型输入

对于自然语言，可通过特征工程的方法定义序列字符特征，如词性特征、前后词等，作为模型输入。现在大多数使用字嵌入或者词嵌入（**embedding**），是事先训练好的或是随机初始化。

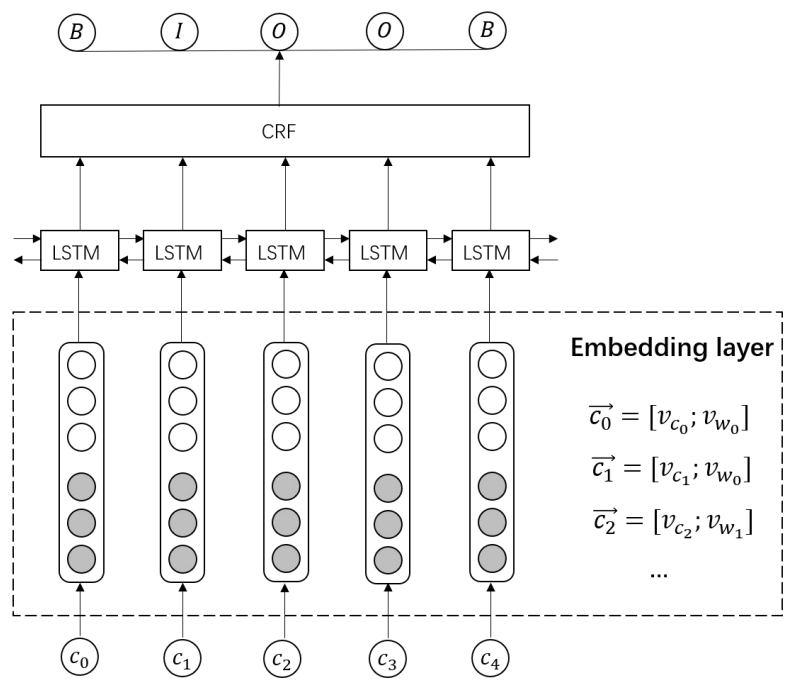


图 4. 模型输入展示

2.2 LSTM 层

LSTM 是一种特殊的 RNN 结构，它解决了传统 RNN 在处理长序列数据时可能遇到的梯度消失或爆炸的问题。LSTM 通过引入门机制（遗忘门、输入门和输

出门)来控制信息的流动和选择,从而实现对长期依赖信息的有效学习。BiLSTM接收每个字符的 **embedding**, 并预测每个字符的对 5 个标注标签的概率。

在 NER 任务中,一般使用一层的双向 LSTM 是足够的。

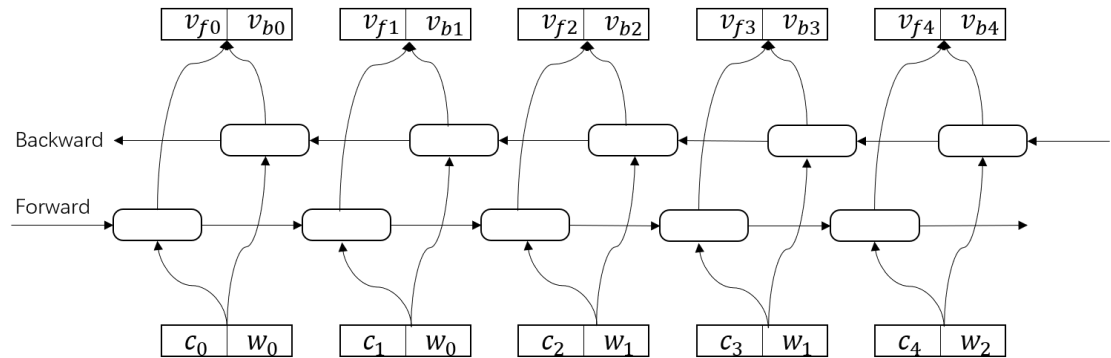


图 5. LSTM 层展示

2.3 全连接层

BiLSTM 输出的向量维度为[num_directions, hidden_size]。为将输入表示为字符对应各个类别的分数,需要在 BiLSTM 层后加入一个全连接层,将其输出映射到一个 5 数值的分布概率。

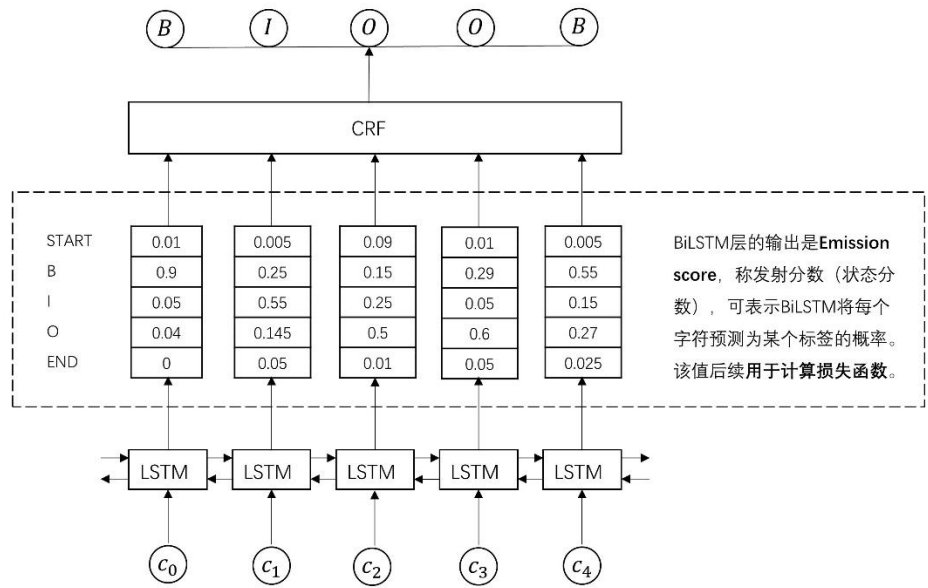


图 6. 全连接层展示

2.4 CRF

CRF 是一种用于建模序列数据的概率图模型,它可以捕获序列中元素之间的局部依赖关系,并给出一种全局最优的标注结果。与 HMM 相比,CRF 没有隐状态,可以直接输出观察序列的最佳标注结果。

设 $X = (x_1, x_2, \dots, x_n)$, $Y = (y_1, y_2, \dots, y_n)$ 为线性链表示的随机变量序列，给定 X 的条件下， Y 的条件概率分布构成条件随机场（即：满足马尔科夫性），称 $P(Y|X)$ 为线性链条件随机场。

其中，输入为 Emission_score，输出为符合标注转移约束条件的、最大可能的预测标注序列。

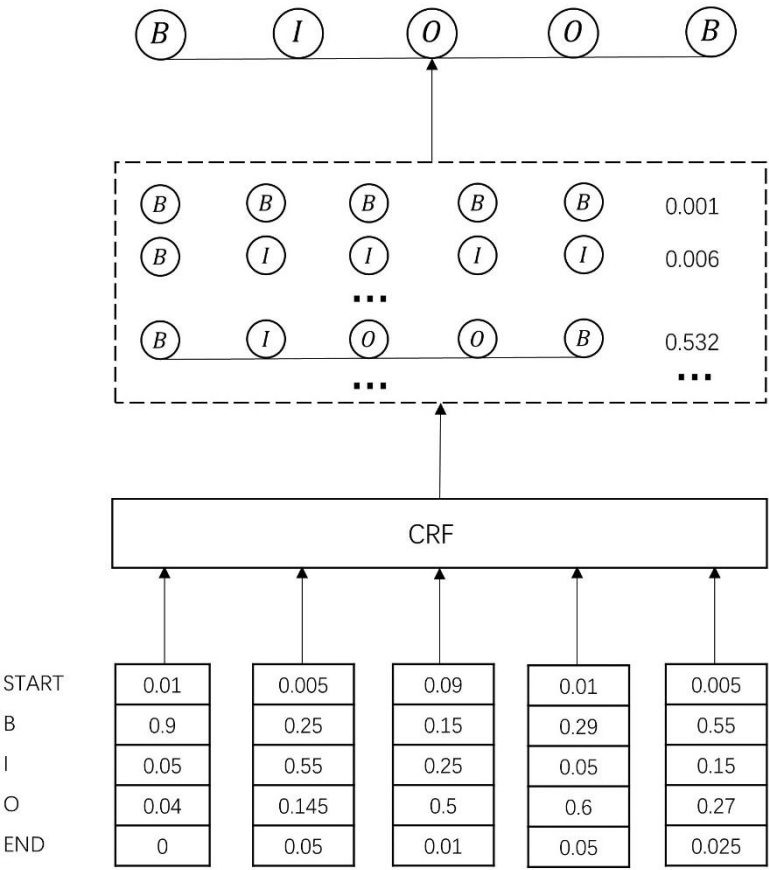


图 7. CRF 展示

2.5 模型整体结构

图 8 展示了模型每一层的细致结构，便于我们理解每一层主要完成的任务以及使用的方法。

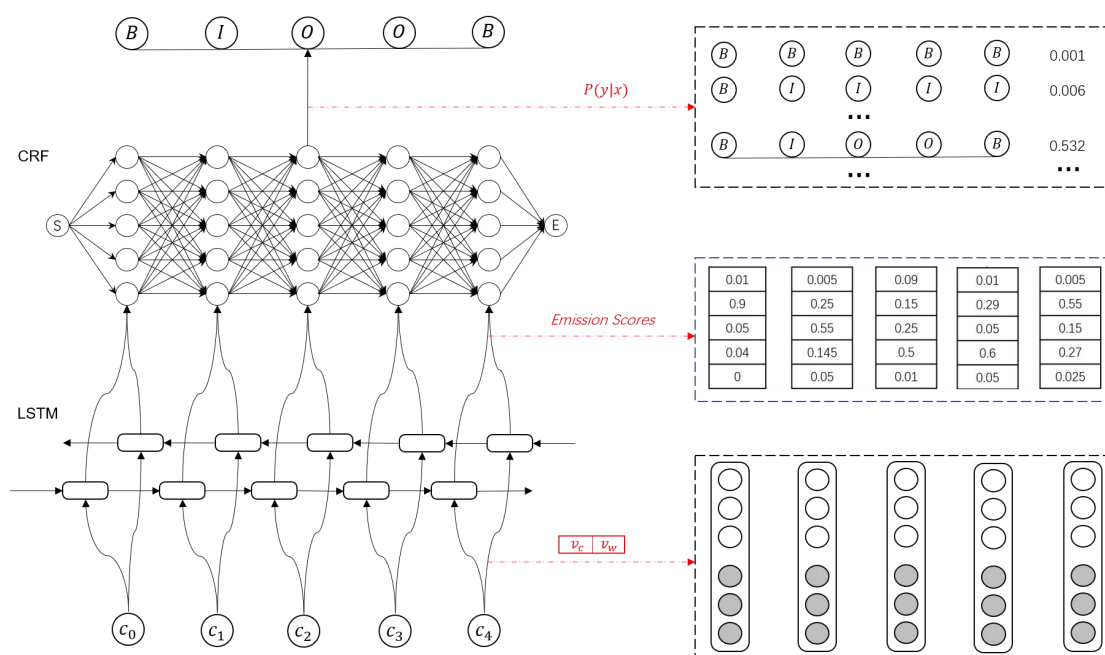


图 8. 模型整体结构

3. BiLSTM-CRF 模型优势

该模型综合 LSTM 和 CRF 的优势，LSTM+CRF 模型在 NER 任务中表现出较高的准确度和效率。LSTM 处理输入序列并捕获其中的特征和依赖关系，然后 CRF 层利用这些特征并通过全局优化来正确标注出整个序列的实体标签。

这种结合方式允许模型不仅仅依赖于单个词语的上下文信息，而且还能够考虑到整个标记序列的合理性，显著提高了命名实体识别的性能。

三、 实验流程

1. 使用 conda 创建虚拟环境

点击 Anaconda Powershell Prompt，打开 powershell。

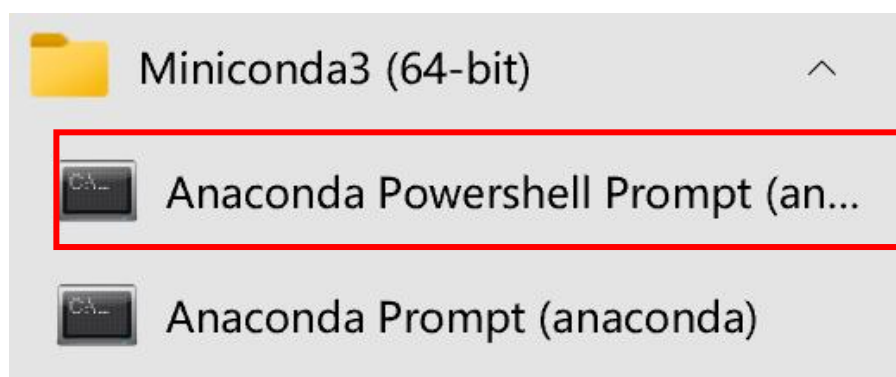


图 9. powershell 打开入口

使用命令 “conda create -n lab1 python=3.8” 创建需要的环境，然后使用命令 “conda activate lab1” 激活环境，之后进入代码所在文件夹。

```
Anaconda Powershell Prompt (anaconda)
(base) PS C:\Users\admin> conda create -n lab1 python=3.8
Channels:
- defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: D:\software\anaconda\envs\lab1

  added / updated specs:
    - python=3.8

The following NEW packages will be INSTALLED:

ca-certificates      pkgs/main/win-64::ca-certificates-2024.3.11-haa95532_0
libffi               pkgs/main/win-64::libffi-3.4.4-hd77b12b_0
openssl              pkgs/main/win-64::openssl-3.0.13-h2bbff1b_0
pip                  pkgs/main/win-64::pip-23.3.1-py38haa95532_0
python               pkgs/main/win-64::python-3.8.19-h1aa4202_0
setuptools           pkgs/main/win-64::setuptools-68.2.2-py38haa95532_0
sqlite               pkgs/main/win-64::sqlite-3.41.2-h2bbff1b_0
vc                   pkgs/main/win-64::vc-14.2-h21ff451_1
vs2015_runtime       pkgs/main/win-64::vs2015_runtime-14.27.29016-h5e58377_2
wheel                pkgs/main/win-64::wheel-0.41.2-py38haa95532_0

Proceed ([y]/n)? y

Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#     $ conda activate lab1
#
# To deactivate an active environment, use
#
#     $ conda deactivate

(base) PS C:\Users\admin> conda activate lab1
(lab1) PS C:\Users\admin> cd D:\DDD\大三下\智能搜索与问答\lab1
(lab1) PS D:\DDD\大三下\智能搜索与问答\lab1> _
```

图 10. Lab1 环境配置

2. 运行实验

输入命令“pip install torch”，安装实验运行所需要的库“torch”。


```
(lab1) PS D:\DDD\大三下\智能搜索与问答\lab1> pip install torch
Collecting torch
  Using cached torch-2.3.0-cp38-cp38-win_amd64.whl.metadata (26 kB)
Collecting filelock (from torch)
  Using cached filelock-3.13.4-py3-none-any.whl.metadata (2.8 kB)
Collecting typing-extensions>=4.8.0 (from torch)
  Using cached typing_extensions-4.11.0-py3-none-any.whl.metadata (3.0 kB)
Collecting sympy (from torch)
  Using cached sympy-1.12-py3-none-any.whl.metadata (12 kB)
Collecting networkx (from torch)
  Using cached networkx-3.1-py3-none-any.whl.metadata (5.3 kB)
Collecting Jinja2 (from torch)
  Using cached Jinja2-3.1.3-py3-none-any.whl.metadata (3.3 kB)
Collecting fsspec (from torch)
  Using cached fsspec-2024.3.1-py3-none-any.whl.metadata (6.8 kB)
Collecting mkl<=2021.4.0,>=2021.1.1 (from torch)
  Using cached mkl-2021.4.0-py2.py3-none-win_amd64.whl.metadata (1.4 kB)
Collecting intel-openmp==2021.* (from mkl<=2021.4.0,>=2021.1.1->torch)
  Using cached intel_openmp-2021.4.0-py2.py3-none-win_amd64.whl.metadata (1.2 kB)
Collecting tbb==2021.* (from mkl<=2021.4.0,>=2021.1.1->torch)
  Using cached tbb-2021.12.0-py3-none-win_amd64.whl.metadata (1.1 kB)
Collecting MarkupSafe>=2.0 (from Jinja2->torch)
  Using cached MarkupSafe-2.1.5-cp38-cp38-win_amd64.whl.metadata (3.1 kB)
Collecting mpmath>=0.19 (from sympy->torch)
  Using cached mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Using cached torch-2.3.0-cp38-cp38-win_amd64.whl (159.8 MB)
Using cached mkl-2021.4.0-py2.py3-none-win_amd64.whl (228.5 MB)
Using cached intel_openmp-2021.4.0-py2.py3-none-win_amd64.whl (3.5 MB)
Using cached tbb-2021.12.0-py3-none-win_amd64.whl (286 kB)
Using cached typing_extensions-4.11.0-py3-none-any.whl (34 kB)
Using cached filelock-3.13.4-py3-none-any.whl (11 kB)
Using cached fsspec-2024.3.1-py3-none-any.whl (171 kB)
Using cached Jinja2-3.1.3-py3-none-any.whl (133 kB)
Using cached networkx-3.1-py3-none-any.whl (2.1 MB)
Using cached sympy-1.12-py3-none-any.whl (5.7 MB)
Using cached MarkupSafe-2.1.5-cp38-cp38-win_amd64.whl (17 kB)
Using cached mpmath-1.3.0-py3-none-any.whl (536 kB)
Installing collected packages: tbb, mpmath, intel-openmp, typing-extensions, sympy, networkx, mkl, MarkupSafe, fsspec, filelock, Jinja2, torch
Successfully installed MarkupSafe-2.1.5 filelock-3.13.4 fsspec-2024.3.1 intel-openmp-2021.4.0 Jinja2-3.1.3 mkl-2021.4.0 mpmath-1.3.0 networkx-3.1 sympy-1.12 tbb-2021.12.0 torch-2.3.0 typing-extensions-4.11.0
```

图 11. 安装库“torch”

输入命令“python train.py”运行实验。

四、 实验结果

```
return tensor.normal_(mean, std, generator=generator)
(tensor(2.6907), [1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1])
(tensor(20.4906), [0, 1, 1, 1, 2, 2, 2, 0, 1, 2, 2])
```

图 12. 实验结果

五、 结果分析

在预测阶段，模型的输出结果为(tensor(2.6907), [1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1])，而期望的标签序列为[0, 1, 1, 1, 2, 2, 2, 0, 1, 2, 2]。可以看到，预测的标签序列与期望的标签序列并不完全一致，存在一些差异。

在训练后的结果中，模型的输出为(tensor(20.4906), [0, 1, 1, 1, 2, 2, 2, 0, 1, 2, 2]), 与期望的标签序列完全一致。这表明在训练阶段，模型能够正确地预测出期望的标签序列。

由此可以看出，模型经过训练阶段学习到了正确的模式，提高了模型进行命名实体识别任务的准确性。

六、心得体会

在本实验中，使用神经网络来实现命名实体识别这项任务，在搭建模型过程中，让我对 lstm 和 CRF 有了更加深刻和细致的理解。

该模型能够有效地捕捉词语之间的上下文信息，这对于命名实体识别任务非常重要。通过双向 LSTM 模型，我们可以更好地理解文本中词语之间的关系，从而提高模型的准确性。CRF 层可以考虑标签之间的依赖关系，这有助于模型更好地理解不同标签之间的转移规律，从而提高了模型的性能。通过结合 BI-LSTM 和 CRF，我们可以充分利用上下文信息和标签依赖关系，提高命名实体识别任务的准确率。

在实验过程中，我使用虚拟环境运行代码，这样我对 conda 的使用更加熟练，切身感受到 conda 的便利。同时，及时分析模型的输出结果对于了解模型表现、发现问题和改进模型至关重要。通过对实验结果的分析，我们可以深入了解模型，直观看到训练后模型对于命名实体识别的准确性。

总的来说，基于 LSTM+CRF 的命名实体识别模型在实验中表现出色，但也存在一些挑战和改进空间。通过实验，我们可以不断积累经验，提高对模型的理解，为实际应用中的命名实体识别任务提供更好的解决方案。